Computational Economics

Lecture: Q Functions, Optimality, and Value Function Iteration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **From a Fixed Policy to Policy Improvement**
  - State values, state-action values, and the maze example
  - Greedy choices and the policy improvement theorem
- **Optimality and Policy Iteration**
  - Bellman optimality and the one-step deviation principle
  - Exact policy evaluation and improvement
- **Value Function Iteration**
  - Synchronous updates and convergence
  - Stopping errors and policy recovery

Recap: Computing the Value Function of a Given Policy

Consider a specific deterministic policy σ.

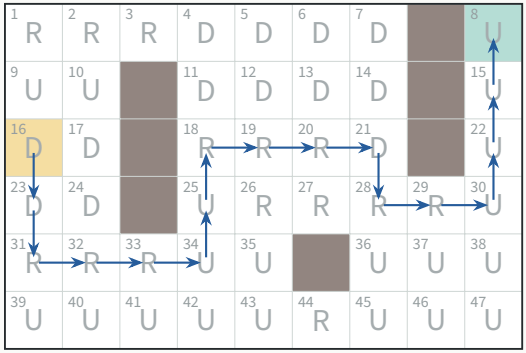


Figure 1: Actions and path under σ.

- U: up; D: down;
  L: left; R: right.
- Each move gives -1, including entry into cell 8.
- Set $\beta = 1$ for simplicity.
- Following this policy reaches cell 8 from every free cell.

Recap: Computing the Value Function of a Given Policy

With $\beta = 1$, compute values backward from cell 8, following the actions of σ.

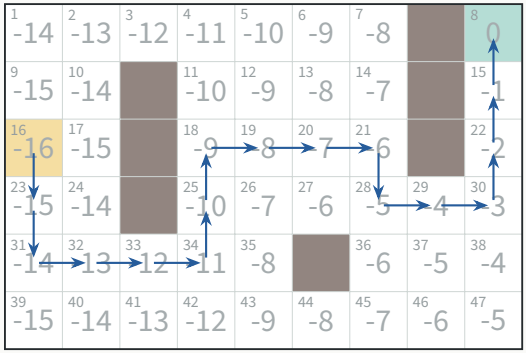


Figure 2: Values and path under σ.

$V^\sigma(8) = 0$. For every other cell,

$$V^\sigma(s) = -1 + V^\sigma(s').$$

$$V^\sigma(15) = -1 + 0 = -1,$$
$$V^\sigma(22) = -1 + (-1) = -2,$$
$$V^\sigma(30) = -1 + (-2) = -3.$$

Continuing backward gives $V^\sigma(16) = -16$.

Each value is minus the number of moves to cell 8 under σ.

From State Values to Action Values

The **state value function** $V^\sigma(s)$: follow the deterministic policy σ now and afterward.

$$V^\sigma(s) = \underbrace{r\big(s, \sigma(s)\big)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}[V^\sigma(s') \mid s,\ a = \sigma(s)]}_{\text{Discounted continuation value}}.$$

The **state-action value function** $Q^\sigma(s, a)$: choose a now, then follow σ.

$$Q^\sigma(s, a) = \underbrace{r(s, a)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}[V^\sigma(s') \mid s, a]}_{\text{Discounted continuation value}}.$$

The first action a need not be the action selected by σ.

If the first action also follows σ, then

$$\text{Deterministic:} \quad V^\sigma(s) = Q^\sigma(s, \sigma(s)),$$

$$\text{Randomized:} \quad V^\sigma(s) = \sum_{a \in \mathcal{A}(s)} \Pr\{\sigma(s) = a\}\, Q^\sigma(s, a).$$

Example: Comparing First Actions

At cell 25, $\sigma(25) = \mathrm{U}$. Compare U and R, then follow σ.

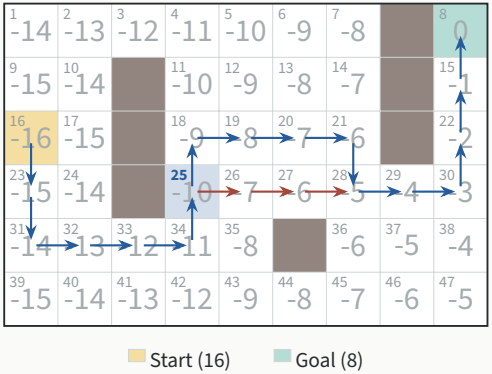


Figure 3: Policy path and shortcut from cell 25.

Up to cell 18:

$$Q^{\sigma}(25, \mathrm{U}) = -1 - 9 = -10.$$

Right to cell 26:

$$Q^{\sigma}(25, \mathrm{R}) = -1 - 7 = -8.$$

The policy chooses U, so $V^{\sigma}(25) = -10$.
Choosing R first gives a higher return.

Greedy Policy Improvement

For a given policy σ, let V^σ and Q^σ denote its state and state–action value functions. At a particular state s, a greedy **policy improvement step** defines the new policy $\widetilde{\sigma}$ by

$$\widetilde{a} \in \arg\max_{a \in \mathcal{A}(s)} Q^\sigma(s,a), \qquad \widetilde{\sigma}(x) = \begin{cases} \widetilde{a}, & x = s, \\ \sigma(x), & x \neq s. \end{cases}$$

i.e., the new policy uses $\widetilde{a}$ at s and keeps σ at every other state.

Policy improvement theorem. More generally, let σ and $\widetilde{\sigma}$ be any two stationary deterministic policies. Assume $|r(s,a)| \leq \bar{r} < \infty$ for all feasible (s,a), and $0 < \beta < 1$. If

$$Q^\sigma(s, \widetilde{\sigma}(s)) \geq V^\sigma(s) \qquad \forall s \in \mathcal{S},$$

then

$$V^{\widetilde{\sigma}}(s) \geq V^\sigma(s) \qquad \forall s \in \mathcal{S}.$$

If, at every state s, taking $\widetilde{\sigma}(s)$ once and then following σ is no worse than following σ throughout, then following $\widetilde{\sigma}$ at every step is also no worse.

The Principle of Optimality

Let σ^* be an **optimal policy**, with associated values

$$V^*(s) = V^{\sigma^*}(s) = \max_{\sigma} V^{\sigma}(s), \qquad Q^*(s,a) = Q^{\sigma^*}(s,a).$$

Principle of optimality: an optimal policy must also be optimal for every remaining **subproblem**. After reaching s', the same σ^* solves the remaining problem starting at s', with value $V^*(s')$.

1. Fix the first action a. Receive $r(s,a)$, then solve the next-state subproblem optimally. Averaging over possible next states gives

$$Q^*(s,a) = r(s,a) + \beta \mathbb{E}[V^*(s') \mid s,a].$$

2. Choose the best first action. The optimal policy σ^* attains the maximum:

$$V^*(s) = Q^*(s, \sigma^*(s)) = \max_{a \in \mathcal{A}(s)} Q^*(s,a).$$

Substituting the first equation into the second gives the **Bellman optimality equation**:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \{ r(s,a) + \beta \mathbb{E}[V^*(s') \mid s,a] \}.$$

Optimality in the Remaining Subproblems

Each dashed box starts a new subproblem at s'. The continuation of σ^* solves it optimally, giving $V^*(s')$.

$$Q^*(s,a) = r(s,a) + \beta \mathbb{E}[V^*(s') \mid s, a]$$

$$V^*(s) = Q^*(s, \sigma^*(s)) = \max_a Q^*(s,a)$$

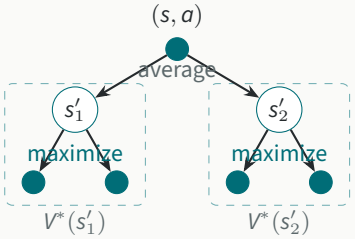


Figure 4: Fix a; continue optimally.

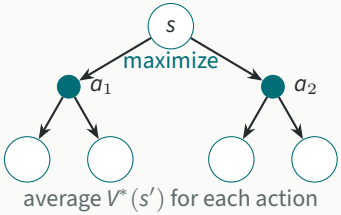


Figure 5: Choose the best first action.

Adapted from Sutton and Barto (2018), Figure 3.4; branches show possible next states.

The Bellman Optimality Operator

The Bellman equation characterizes V^*, but V^* is still unknown. To compute it, start with a candidate continuation-value function V.

The **Bellman optimality operator** T maps V to an updated value function TV:

$$(TV)(s) = \max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V(s') \mid s, a]\}.$$

At each state, evaluate each first action using V for the continuation, then keep the highest value.

At the optimal value, the update changes nothing:

$$TV^* = V^*.$$

Thus V^* is a **fixed point** of T.

This suggests repeated updates, $V_{k+1} = TV_k$. We need to establish that they converge to V^*, regardless of the initial guess V_0.

Convergence of Bellman Updates

To justify repeated updates, compare how T transforms two candidate value functions V_1, V_2. Their largest difference across states is

$$\|V_1 - V_2\|_\infty = \max_s |V_1(s) - V_2(s)|.$$

Averaging cannot increase this difference; discounting multiplies it by β. Maximizing over actions preserves the bound:

$$\|TV_1 - TV_2\|_\infty \leq \beta \|V_1 - V_2\|_\infty, \qquad 0 < \beta < 1.$$

This is the **contraction property**: the largest difference after an update is at most β times its previous size.

The contraction mapping theorem therefore gives:

- **A unique solution.** V^* is the only fixed point of T, hence the only solution to the Bellman optimality equation.
- **Convergence.** Starting from any V_0, the updates $V_{k+1} = TV_k$ satisfy

$$\|V_k - V^*\|_\infty \leq \beta^k \|V_0 - V^*\|_\infty \longrightarrow 0.$$

When Can Policy Improvement Stop?

Policy improvement raises values. To turn it into an algorithm, we need a condition that tells us when the current policy is optimal.

The **one-step deviation principle** provides this stopping condition:

$$\sigma(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^\sigma(s,a) \quad \forall s \quad \iff \quad V^\sigma = V^*.$$

If no action gives a higher Q^σ at any state, then

$$V^\sigma(s) = \max_{a \in \mathcal{A}(s)} Q^\sigma(s,a) = (TV^\sigma)(s).$$

Thus V^σ is a fixed point of T. Uniqueness implies $V^\sigma = V^*$.

Stopping rule for policy iteration: if the greedy improvement step leaves the policy unchanged, stop; the current policy is optimal.

Policy Iteration

Start from any feasible policy σ_0. Repeat:

1. **Policy evaluation:** solve exactly for the current policy's value,

$$V^{\sigma_k}(s) = r(s, \sigma_k(s)) + \beta \mathbb{E}[V^{\sigma_k}(s') \mid s, a = \sigma_k(s)].$$

2. **Policy improvement:** choose

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a), \qquad \forall s.$$

   Keep the current action whenever it is a maximizer.

3. If $\sigma_{k+1} = \sigma_k$, stop: the policy is optimal. Otherwise evaluate the new policy and repeat.

$$V^{\sigma_0} \leq V^{\sigma_1} \leq V^{\sigma_2} \leq \cdots \leq V^*.$$

Each nonoptimal policy improves strictly at some state. With finitely many policies, this process terminates at an optimum.

Value Function Iteration: Inputs and Updates

0. **Input.** State set $\mathcal{S}$, feasible actions $\mathcal{A}(s)$, rewards $r(s,a)$, transition probabilities $P(s' \mid s,a)$, discount factor β, and target value error $\varepsilon > 0$.
1. **Initialize.** Set $k = 0$ and $V_0(s) = 0$ for every $s \in \mathcal{S}$.
2. **Compute action values.** For every $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, compute
$$Q_k(s,a) = r(s,a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V_k(s').$$
Use the same V_k for all states and actions.
3. **Update state values.** For every $s \in \mathcal{S}$, compute
$$V_{k+1}(s) = \max_{a \in \mathcal{A}(s)} Q_k(s,a).$$
Store V_{k+1} separately; keep V_k for the stopping check.

Value Function Iteration: Stopping and Outputs

4. **Measure the update.** Compute the **Bellman residual**:
$$d_k = \max_{s \in \mathcal{S}} |V_{k+1}(s) - V_k(s)|.$$
5. **Check the tolerance.**
   - If $d_k > (1-\beta)\varepsilon$, set $k \leftarrow k+1$ and return to **Step 2**.
   - Otherwise, continue to **Step 6**.
6. **Extract the policy.** For every $s \in \mathcal{S}$, select
$$\widehat{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q_k(s,a).$$
   Use a fixed action order to break ties.
7. **Return and stop.** Output
$$\widehat{V} = V_k, \qquad \widehat{\sigma}.$$
   The returned value is the one used to compute Q_k and satisfies $\|\widehat{V} - V^*\|_\infty \leq \varepsilon$.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 3: Finite Markov Decision Processes
   - Chapter 4: Dynamic Programming
2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — The Bellman Equation and the Principle of Optimality
   - Discrete State Dynamic Programming — Policy Function Iteration
   - Discrete State Dynamic Programming — Value Function Iteration